

StarlingMonkey TNG

Introducing a new generation of StarlingMonkey

Till Schneidereit · Bytecode Alliance Plumbers Summit · 2026-10-28

WHY A REWRITE

Four goals for the new runtime

Safety by construction

GC rooting and object state are checked by the Rust compiler, not by review.

WASIp3 first

Async exports, streams and futures, and wasi:http 0.3, with nothing that blocks.

WIT that feels like JavaScript

Streams, promises, classes and exceptions instead of handles and tags.

Extensible and maintainable

Extensive infrastructure for building additional builtins at scale

USING IT

Natively, dynamically, and wizened

Develop natively	`starlingmonkey app.js`
Run JS in wasm dynamically	`wasmtime run --dir=. -Shttp=y starling.wasm app.js`
Build a component	`starling-componentize --export-http componentize app.js -o app.wasm`
Serve it	`wasmtime serve app.wasm`

USING IT

Serving HTTP

```
// app.js, exported as wasi:http/handler
export async function handle(request) {
  if (request.getPathWithQuery() === "/proxy") {
    return fetch(request);  // forwards the request unchanged
  }
  return new Response`hello from ${request.getPathWithQuery()}`);
}
```

Or register addEventListener("fetch", ...).

Start from a world

```
package demo:shop@0.1.0;

interface inventory {
  record item { sku: string, name: string, cents: u64 }
  variant lookup-error { not-found(string), unavailable }
  lookup: async func(sku: string) -> result<item, lookup-error>;
}

interface catalog {
  price-list: async func(skus: stream<string>) -> stream<string>;
}

world shop { import inventory; export catalog; }
```

Imports become ES modules the app can import.

Exports are implemented by the app's main module.

Async, streams and futures are mapped to idiomatic JS types.

Generate TypeScript declarations for the world

```
starling-componentize -w wit -n shop types -o world.d.ts
```

```
import type * as Guest from "starling:guest";
import { lookup, LookupError } from "demo:shop/inventory@0.1";

export const catalog: typeof Guest.catalog = { /* checked by tsc */ };
```

starling:guest declares every shape the world allows for each export.

Implement it in JavaScript

```
import { lookup, LookupError } from "demo:shop/inventory@0.1";

async function* priceList(skus) {              // skus: a ReadableStream
  for await (const sku of skus) {
    try {
      const { name, cents } = await lookup(sku);  // cents: a BigInt
      yield `${sku},${name},${cents}`;
    } catch (e) {
      if (!(e instanceof LookupError)) throw e;
      yield `${sku},,${e.payload.tag}`;          // "notFound" or "unavailable"
    }
  }
}

export const catalog = { async priceList(skus) { return priceList(skus); } };
```

CUSTOM WIT · 4 OF 4

Build it, then compose it like any component

```
# build the component from app.js and the world
starling-componentize -w wit -n shop componentize app.js -o shop.wasm

# plug in any component that exports demo:shop/inventory
wac plug shop.wasm --plug inventory.wasm -o app.wasm
```

Exactly the world's type

The component exports what the world declares, and nothing else.

Minimal imports

Only what the world and the runtime's builtins use.

Snapshotted

The app's top level runs at build time, then Wizer takes a snapshot.

WIT <-> JS Mapping

WIT	JavaScript
u64, s64	BigInt
list<u8>, list<f64>, ...	Typed arrays (plain arrays are accepted too)
record, variant	Objects with lowerCamelCase fields, variants as { tag, val }
enum, flags	Numbers, with a frozen name map such as Qos.atLeastOnce
resource	A class, and [Symbol.dispose]() releases the handle
result<T, E>	A return value, or a thrown subclass of ComponentError
stream<T>	A ReadableStream coming in, any (async) iterable going out
future<T>	A Promise

EXAMPLE · STREAMS

WIT streams and fetch bodies are the same thing

```
// download: async func(url: string) → stream<u8>
// upload: async func(url: string, body: stream<u8>) → u16
export const files = {
  async download(url) {
    const response = await fetch(url);
    return response.body;
  },
  async upload(url, body) {
    const response = await fetch(url, { method: "POST", body });
    return response.status;
  },
};
```

EXAMPLE · RESOURCES

Resources are classes, in both directions

Imported: release handles with using

```
import { Connection } from "demo:db/client@0.1";

export async function rowCount(table) {
  using conn = new Connection("local");
  return conn.count(table);
}  // handle released here
```

Exported: implement a class

```
export class Session {
  constructor(user) { this.user = user; }
  name() { return this.user; }

  [Symbol.dispose]() {
    // called when the host drops it
  }
}
```

EXAMPLE · ERRORS

WIT results are mapped to JS exceptions

```
import { lookup, LookupError } from "demo:shop/inventory@0.1";

// an import's err is thrown as an instance of the type's class
try {
  await lookup("sku-42");
} catch (e) {
  if (e instanceof LookupError) console.log(e.payload);  // { tag: "notFound", val: "sku-42" }
}

// reserve: async func(sku: string) → result<u32, lookup-error>
export async function reserve(sku) {
  throw new LookupError({ tag: "unavailable" });  // becomes err(unavailable)
}
```

DESIGN

Layering on top of SpiderMonkey

starling	The starlingmonkey CLI and the starling.wasm runtime component
libstarling	CLI, and serving natively and on Wasm
platform	HTTP and clocks for each host
builtins/*	fetch, streams, URL, globals and fetch events
wit-bridge	Lifts and lowers WIT values for JS
core-runtime	Runtime, event loop, timers and the module loader
js	The safe SpiderMonkey API, and the only crate that uses mozjs
SpiderMonkey	Through the mozjs crate

DESIGN

Builtins defined using proc macros

```
#[jsclass]
struct Counter { value: i32 }

#[jsmethods]
impl Counter {
    #[constructor]
    fn new(initial: i32) -> Self { Self { value: initial } }

    #[method]
    fn increment(&mut self) { self.data_mut().value += 1; }

    #[getter]
    fn value(&self) -> i32 { self.data().value }
}
```

Rooting in the type system

Scope, Stack and Heap types make safe GC handling viable.

State handling

All instance state lives in a Rust struct

WebIDL as an attribute

#[webidl_interface] gives a class WebIDL semantics.

DESIGN

From a world and a module to a component

1
Load
wit-parser reads the world, merging packages and matching versions

→

2
Bind
wit-dylib generates a small bindings module for the world

→

3
Link
Static linking with the prebuilt starling.wasm

→

4
Snapshot
Wizer runs the app's top level and saves the memory

→

5
Compose
wac-graph builds the final component with the world's type

> At run time, wit-bridge lifts and lowers values from the world's type descriptions, and provides each imported interface as an ES module.

DESIGN

Async on WASIp3, without blocking

An event loop per async call

fetch, timers and streams work inside an async export. The call ends when its promise and its work are done.

Sync exports have no loop

Microtasks run before they return. Starting a timer throws a TypeError naming the export.

Instances are reused

Hosts send many requests to one instance, so builtin state is per instance, never per request.

Nothing blocks

No block_on anywhere. All I/O uses WASIp3 streams and futures.

PRELIMINARY BENCHMARKS

WIT calls cost a fraction of what they did

61×

faster per call than ComponentizeJS on average.

Call	ComponentizeJS	Starling	Speedup
Export, no arguments	68.8 µs	0.23 µs	303×
Export, a u32	68.4 µs	0.31 µs	223×
Export, a short string	74.8 µs	0.43 µs	172×
Import, no arguments	19.6 µs	0.23 µs	87×
Import, a long string	371 µs	3.5 µs	106×
Export, a list of records	1,055 µs	106 µs	10×

PRELIMINARY BENCHMARKS

Against the C++ runtime, both on Wasm

1.9×

Web builtins

Geomean of 53 microbenchmarks, 37 of them faster. Request cloning up to 10.9×, Response.json 6.9×.

1.0×

Serving, a fresh instance per request

Geomean of 7 handlers. JSON 1.36× faster, streaming 0.59×.

2.2×

Serving with instance reuse

Geomean of 7 handlers. Hello world goes from 5.4k to 15.6k requests/s, and p50 latency from 181 to 61 µs.

WHAT'S NEXT

What remains to be done

- Make it official!
- WinterTC compatibility
- NightMonkey integration
- Node.js API layer
- WASIp2 backend
- Debugging support



BYTECODE
ALLIANCE

Thank you

github.com/bytecodealliance/StarlingMonkey-ng